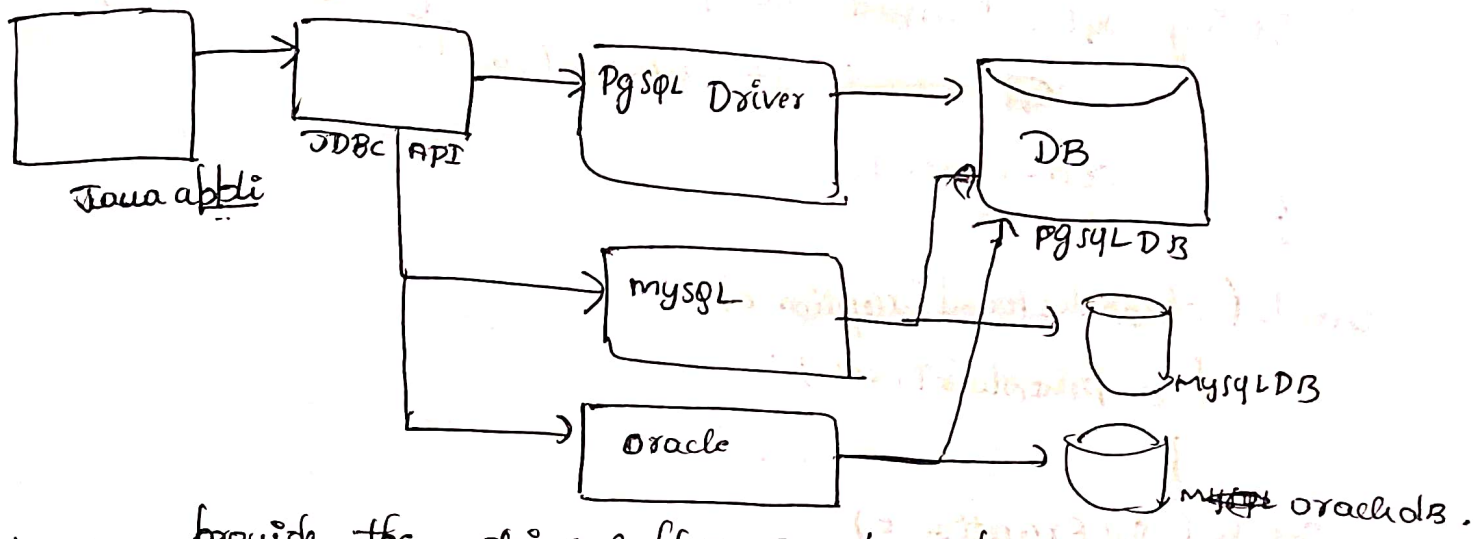


JDBC API :-

- * It helps to connect an execute query with the db.
- * JDBC API has provided from Sunmicrosystem.

How java application interact with db?



Who will provide the driver software :- db vendors

by only JDBC API can't connect the Java application & DB so we need 'Driver' (it is independent) by using API & Driver to connect the Java application & Database.

To do any Crud operation in JDBC

① Load / Register the driver

② Create Connection.

③ Create Statement

④ Execute Query

⑤ Close the Connection

Ex:-

```
class Demo {  
    {  
        psum() {  
            try {
```

```

class.forName("org.postgresql.Driver");
Connection con = DriverManager.getConnection(url, user, password);
Statement st = con.createStatement();
String sql = "Insert into product values(1, 'Rish', 10, 2, 2);";
sql Execute st.executeUpdate(sql);
con.close();
}
catch (ClassNotFoundException e)
{
    e.printStackTrace();
}
catch (SQLException e)
{
    e.printStackTrace();
}
}

```

① To Load / Register the driver

* the method forName() which is present inside the class we are using the postgresql to load the driver software. i.e. org.postgresql.Driver

Load → `class.forName("org.postgresql.Driver");`

Register → `Driver driver = new Driver();` → create a object of driver.
`DriverManager.registerDriver(driver);` ^{class}

forName() * public static method, so we use class name to call
 * It accept the parameters of fully qualified driver software
 * It is present in java.lang
 * It throws ClassNotFoundException

Driver is built in class
 ↳ It is present org.postgresql
 ↳ It throws SQL Exception

```
product
int id
String name
int price
int quality
```

② Create a Connection:-

Connection con = DriverManager.getConnection("url", "user", "password");
getConnection is a public static method SQL Exception
 * It is present inside the DriverManager

Connections It is interface, so use implemented class
 "url = jdbc:postgresql://localhost:5432/shop"
 username = "postgres"
 password = "root"

3 methods :-
 getConnection(url, user, pass)
 getConnection(url)
 getConnection(url, Properties) - is properties

③ Create a prepare Statement

Statement
 PreparedStatement

PreparedStatement pstmt = con.prepareStatement("insert into product values
 (?, ?, ?, ?);"
 ↳ is delimiter

pstmt.setInt(1, 1);
 pstmt.setString(2, "pen");
 pstmt.setInt(3, 10);
 pstmt.setInt(4, 1);

→ Statement st = con.createStatement();
 String sql = "logic";

④ Execute Query

pstmt.executeUpdate();
 st.executeQuery();

Statement → statically giving values
 PreparedStatement → taking input dynamically
 ↳ use setX() method to pass values
 (?, ?, ?)

⑤ close the Connection:- con.close()

ResultSet :- whenever we want to retrieve the data

↳ Set of Result
 ↳ Execute the select query use ResultSet

Statement st = con.createStatement();

String sql = "select * from product";

ResultSet rs = st.executeQuery(sql)

while (rs.next())
 {
 rs.getInt(1); rs.getInt(2);
 rs.getString(2); rs.getInt(4);
 } con.close();

③ Execute methods

- 1) boolean Execute() → it is used for insert query
- 2) ResultSet ExecuteQuery() → it is used for select query
- 3) int ExecuteUpdate() → it is used for update query, delete query
→ It returns how many rows affected

Program :-

```
class Demo {
```

```
    main() {
```

```
        try {
```

```
            Class.forName("org.postgresql.Driver");
```

```
            Connection con = DriverManager.getConnection(
                "jdbc:postgresql://localhost:5432/shop", "postgres", "root");
```

```
            Statement st = con.createStatement();
```

```
            String sql = "insert into product values(4, 'Candy', 10, 5)";
```

```
            int rows = st.executeUpdate(sql);
```

```
            con.close();
```

```
            S.o.pln(rows);
```

```
        } catch (ClassNotFoundException | SQLException e) {
```

```
            e.printStackTrace();
```

```
        }  
    } catch (SQLException
```

Close :- we have to close our Connection, otherwise we may lose our data

* close() method is expensive (in the form of data)

* we have one more way to close the Connection without calling

```
try (Connection con = DriverManager.getConnection( )) {  
    }  
    put your connection inside the try block
```


Batch Execution:-

- * Whenever we want to store multiple data in our database in a single shot we will go for batch Execution.
- * To perform batch Execution we need one method call `addBatch()` which will work like a container.
After Statement Execute
`stmt.addBatch()` → prepared statement
→ `stmt.addBatch("insert into ...")`
→ `stmt.executeBatch()`
- * To Execute batch we have one method `executeBatch()` the return type of `executeBatch` is `int[]` (int array).

Transaction:-

- * To store the data in the database when based on the given Condition

① `Con.setAutoCommit (false);`

② After Execute the statement

→ ① `Con.commit()` → if (success)

→ ② `Con.rollback()` if (not success) → store the last transaction.

→ ③ `Savepoint()` (`Con.setSavepoint()`);

Examples:-

booking-info

id	name
1	smith

passenger-info

id	name
1	smith
2	martin

payment-info

id	name
1	smith

Here if the payment is successful then only passenger-info store the data, and if but booking-info should store the data if transaction is failed.

use transaction methods { `commit()`;
`rollback()`;
`savepoint()`;

```
class Demo {
```

```
    @ psum() {
```

```
        try {
```

```
            Class.forName("org.postgresql.Driver");
```

```
            Connection con = DriverManager.getConnection(url, user, password);
```

```
            Statement stm = con.createStatement();
```

```
            String sql = "Insert into booking-info values(1, 'Smith');"
```

```
            stm.execute(sql);
```

```
            Savepoint s = con.setSavepoint();
```

only this data should store

```
            Statement stm1 = con.createStatement();
```

```
            String
```

```
            sql1 = "Insert into passenger-info values(1, 'Smith');"
```

```
            stm1.execute(sql1);
```

```
            if (Getpayment.isSuccess())
```

```
            {
```

```
                "Insert into payment-info values(1, 'Smith');"
```

```
                con.commit();
```

```
                sopln("Success");
```

```
            } else {
```

```
                con.rollback(s);
```

```
                con.commit();
```

```
                sopln("unsuccessful");
```

```
            con.close();
```

```
        }
    }
}
```

```
class Getpayment {
```

```
    { public static boolean isSuccess
```

```
        { return true;
```

```
        }
```

```
    }
```


Web application:- program runs on server & opens in browser with internet.

actions:- along with endpoints take a input

<form action="save"> </form>

HttpServlet → for taking request & response from client

@webServlet :- ~~is~~ It is class level annotation, belongs to ServletApi

* It is required to map the client request to java class

* In Servlet class take a request & provide the service

* Override the service method

Jsp:- we can write html, CSS, java, javascript also.

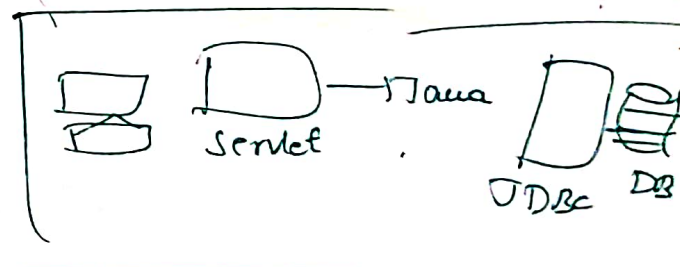
↳ <% %> → scriptlet tag (to write java program)

<%= %> → Expression tag

Servlet life cycle :-

- ① ^{Servlet} Class loading ^{phase} → static blocks executed at the time of
- ② ^{Object} Creation ^{phase} → non static blocks, Constructors will execute.
- ③ Initialization (init() invoking) phase
- ④ Service() ^{invoking} phase
- ⑤ Destruction phase → Destroy() invocation.
↳ whenever you stop the server

Drawback:-



- ① Duplicate Code → JDBC
- ② I/O not generated (pk)
- ③ Tables are not created automatically.
- ④ SQL is DBMS dependent → I want DBMS Independent
- ⑤ JDBC API only works with literals unable to deal with objects
- ⑥ No temporary memory to execute queries

↓
No data
String data

Drawback of JDBC :-

* JDBC API follows same steps for all DB operation (CRUD) as below

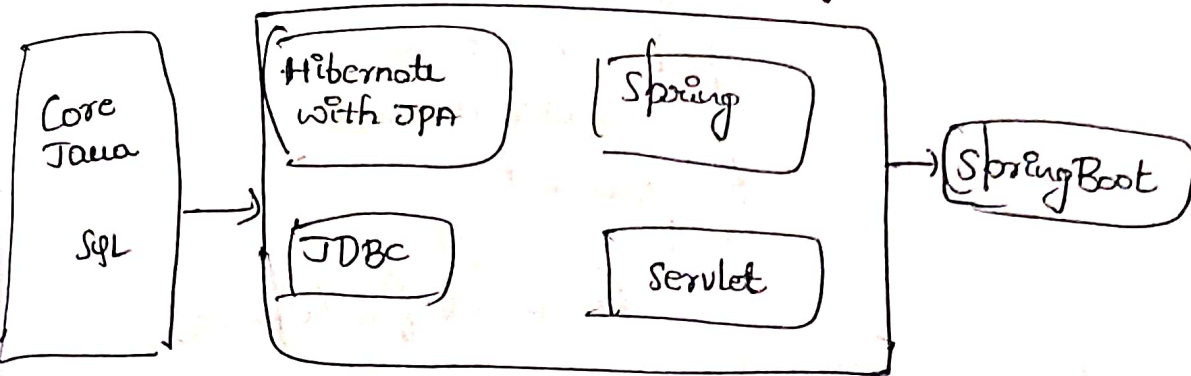
- ① Load / Register DB
- ② Create Connection.
- ③ Create Statement
- ④ Execute the Query/Statement
- ⑤ process the Resultant data
- ⑥ Close ~~the~~ all the costly resources

All above steps are has to be followed to perform the DB operations which creates Code duplicacy in application just duplicate Code is technically referred as Boilerplate Code

- ① JDBC API Can't autogenerate the Primarykey values.
- ② JDBC API Can't Create Table automatically.
- ③ JDBC API : Doesn't Supports objects ~~and~~, it only deals with literals so we lost the advantages of oops principles
- ④ JDBC API works Structured query language which is DBMS dependent Syntax, will be there which is not a recommended thing in a real time application.
- ⑤ JDBC API doesn't Supports Cache memory hence the Performance of the application will decrease.

Servlet :-

* Advance Java is a platform where we can develop Scalable, Robust, Businessgrade application.



* ~~IA DBMS~~ In flat file store or Storage System can store the data permanently but it has some limitation

1) Data Duplicates

2) Limited Capacity which affect in real time use cases

3) It doesn't provide the Constraints

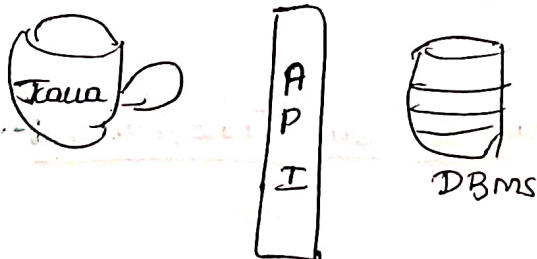
4) It doesn't provide a structured query language to over

come the drawbacks we can use DBMS.

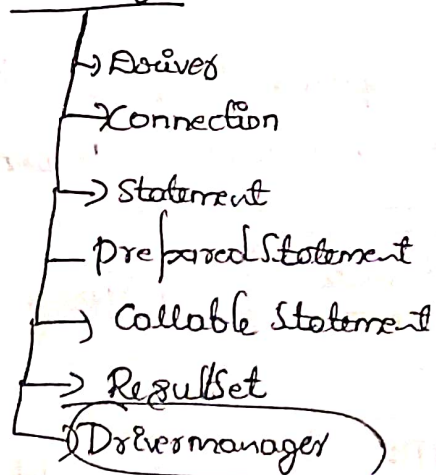
⇒ To Communicate from Java to DBMS we need API

API (Application programming Interface)

It is used to Achieve InterApplication Communication.



* In JDBC API we have some Interfaces



↳ not interface helper class

* Tight Coupling:- Change in the implementation which effect the user is called tight Coupling.

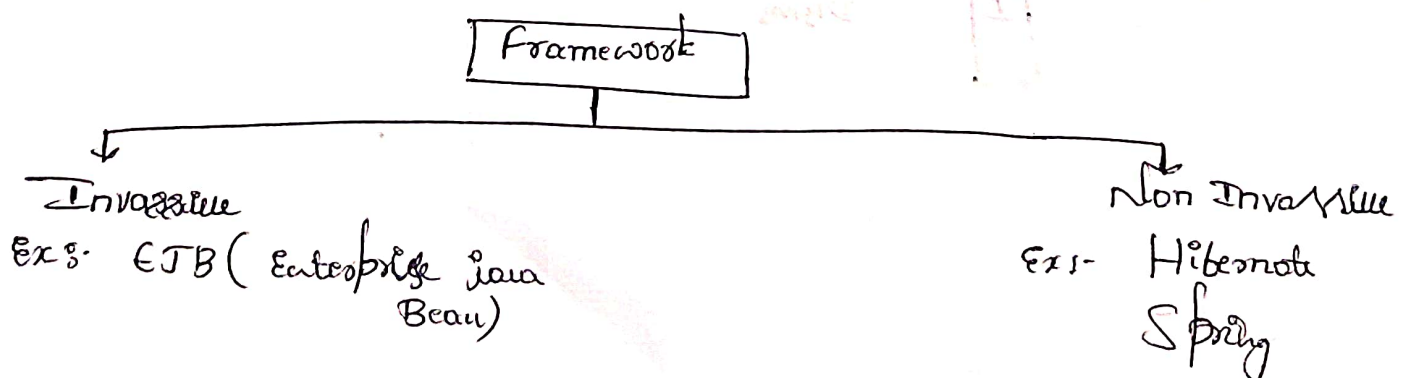
* Loose Coupling:- Change in the implementation doesn't effect the user

FRAMEWORKS :-

- Framework is a hierarchy of classes & interfaces
- Framework is a body (or) platform of predefined code using which we can develop over applications, which has a solution for various technical solution problem

Ex:- ArrayList which is growable in nature also it accept both homogenous & heterogenous object, which is solution for normal conventional array.

Classification of Framework based on Inheritance :-



① Invasive :-

Any framework which develop force the developers to inherit to predefined class (or) interfaces

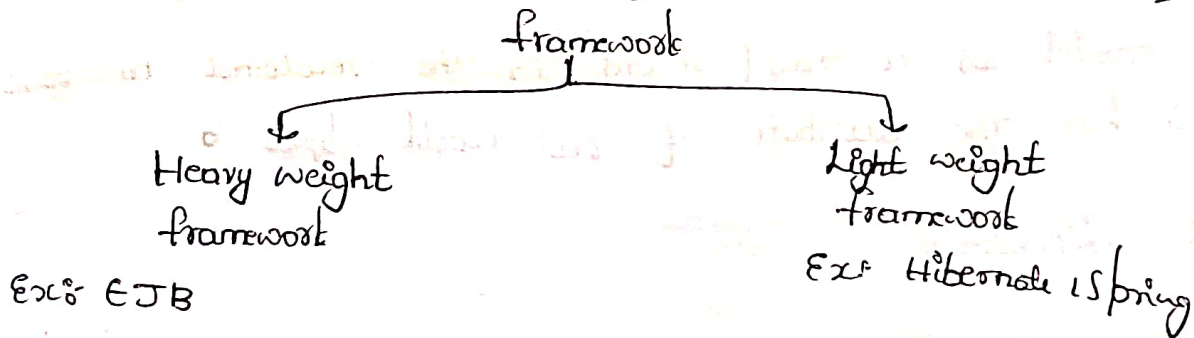
② Non - Invasive :-

Any framework which doesn't force the developer to inherit the predefined class / interface

POJO (plain old Java object) :-

It is a Simple Java class which doesn't have any restriction other than restriction which are imposed by Java, such simple classes we call it as POJO classes. & objects of such class called POJO object.

Classification of framework based on POJO implementation :-



Heavy weight framework :- which doesn't use any POJO class & object
Light - - - - - :- uses - - - - -

Rules to create a POJO class :-

① A POJO class must & should be a public

② POJO class should not extend to any predefined class except Object class

③ POJO class shouldn't implement any predefined interface

④ All the data member in a POJO class must be private

⑤ All the private fields must ~~if~~ should have public getter & setter methods

⑥ Every pojo class must ~~be~~ should contain a zero parameterised constructor mandatorily, if not then should define explicitly

Object Relational Mapping :- (ORM)

It is the process of converting Java model (object) into relational model is called object (ORM).

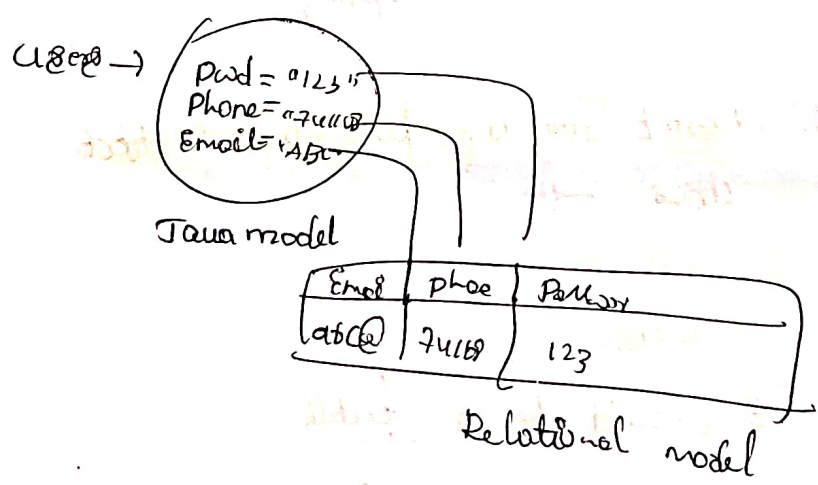
• Java Model :-

Java model refers to a block of memory in the heap area which consist of the data members of an realworld object

• Relational model :-

Relational model is a row / record in the relational database table which has the attribute of real world object

Structure of ORM :-



Specification of ORM

- * In ORM Every class represents a table in the RDBMS
- * In ORM objects of class represents row / records in the table
- * In ORM Every field of the object represents column in the table

Entity :-

- * Entity class is a one which is mapped to the relational database table

Rules to Create a Entity class :-

- * It 'must' & 'should' follow all the rules of POJO class.
- * Every Entity class must & should annotated by @Entity annotation
- * Every Entity class must & should have one field which represents primary key & annotated with @Id

@Entity :-

- It is a class level annotation belongs to "hibernate and Java persistence API (JPA)".

@Entity annotation it indicates that for the annotated class table will be generated & that class will be mapped to the table

@Id :-

@Id is a field level annotation belongs to "hibernate & JPA"

- * It specifies that the annotated field will be act as primary key field.

Ex 8-

@Entity

```
public class User {
```

@Id

```
private int id = 123,
```

```
private String name = "Raghma",
```

```
private String pwd = "123"
```

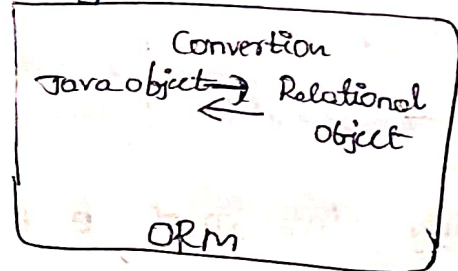
```
}
```

```
public getter & setter ()
```

```
{
```

```
}
```

Why Hibernate



HIBERNATE FRAMEWORK :-

Definition:- Hibernate is a open source, non-invasive, lightweight ORM tool used for object relational mapping.

Advantage of Hibernate

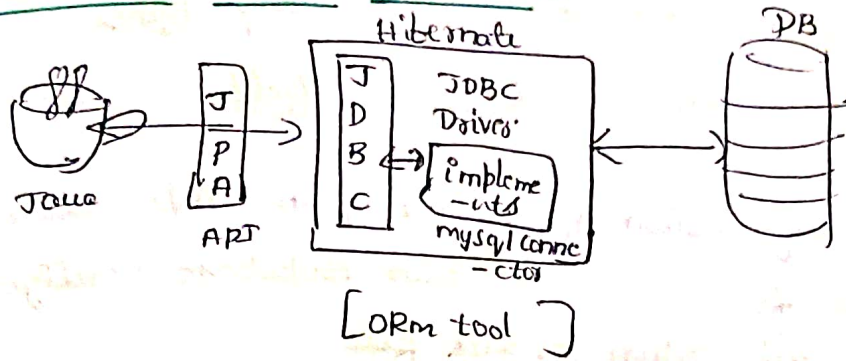
- Hibernate is a Open Source framework
- Hibernate is a non invasive framework
- Hibernate is a lightweight framework
- Hibernate provides predefined function or methods for database operation (CRUD) by which boilerplate code can be avoided.
- Hibernate can autogenerated the primary key values.
- * Hibernate can create the table automatically
- * Hibernate can deal with the objects by which we can utilize object oriented concepts.

* Hibernate provides DB independent query language called Hibernate Query language (HQL). { Easy to change the DB {mysql → postgresql} }

* Hibernate supports Cache a memory by which the performance of the application can be optimized. Note: Cache memory is a temporary memory, which is used to store the data to serve the future request in faster manner.

* Hibernate achieving the relationship b/w tables is easy.

Structure of Hibernate



Note: famous ORM tools

- i) Hibernate
- ii) J-Batis
- iii) Top Link
- iv) My-batis

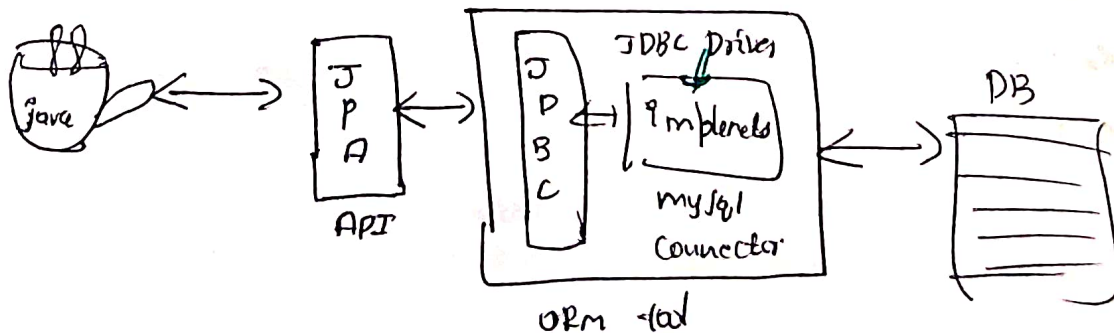
Java Persistence API (JPA)

* It is the specification given in the form of abstraction API to achieve { " Loose Coupling " b/w Java application & ORM tool }.

* For the JPA abstraction API the implementation is ORM tool (Hibernate).

* The advantage of JPA is we can achieve Loose Coupling b/w Java & ORM.

Structure of Hibernate with JPA



Hibernate Query Language (HQL) :-

table → class
row → object
columns → fields

- * It is a powerful database independent Query language.
- * HQL is based on SQL but it is object oriented query language.
- * SQL works with table Hibernate works with Entity.
- * SQL works with rows & Hibernate works with Entity objects.
- * SQL works with Column but HQL works with field.

Note :- xml :- To store the information & to carry the information
persistence.xml → resource file which contain Database Configuration information in xml file.

* which will be utilized by Hibernate & JPA to Communicate DB.

META-INF

↳ persistence.xml (meta information)

present

EntityManagerFactory @ Emf = persistence.createEntityManagerFactory(dev)

EntityManager Em = Emf.createEntityManager();

EntityTransaction Et = Em.getTransaction();

Em.persist(p); product object

Et.commit();

Em.close();

Emf.close();

EntityManagerFactory :-

- * It is an interface belongs to Java persistence API (JPA)
- * This interface object can be created by `createEntityManagerFactory` which is a public static method of persistence class - which will accept a String type argument which should be persistence unit name ("dev").
- * This object is responsible for load & register the drivers & maintaining the connection also to close all the costly resources.
- * `EntityManagerFactory` as a non static method with the name `createEntityManager` using which we can create a object of `EntityManager`.

EntityManager :-

- * It is an interface belongs to Java persistence API (JPA)
 - * The object of `EntityManager` can be created by `createEntity` method of `EntityManagerFactory`.
 - * `EntityManager` is responsible to maintain & manage the Entity objects by performing `create statement` & `execute statement` & processing the result and data.
 - * `EntityManager` has the method to perform database operation (CRUD)
- methods of `EntityManager` are as follows
1. `persist()` → save
 2. `find()` → fetch
 3. `merge()` → save & update
 4. `remove()` → delete
 5. `createQuery()` → custom query execution

EntityTransaction :-

- * It is a interface belongs to Java persistence API
- * The object of `EntityTransaction` can be created by `getTransaction` method of `EntityManager`.

* Entity Transaction object is responsible to manage the DB Transaction. Importantly we will do beginning the transaction & committing the transaction by using below methods.

① begin(); → To begin the transaction.

② commit(); → to commit the transaction.

* Transaction has to begin and commit for the DML operation transaction (Create, update, delete) is mandatory, for DDL operation is optional of begin() & commit().

persist() :- one argument

* It is a non-static method of EntityManager and it will accept an Entity object as a argument which needs to be saved or persistence to the table and it will save the object but returns nothing hence return type of persist method void.

* This method throws illegal argument Exception if we are trying to persist non Entity object.

find() :- 2 argument

It is a non static method of EntityManager which will accept two arguments Entity type as first argument & pk values as 2nd argument.

* This method will find the Entity object based on type mentioned as 1st argument with pk values 2nd argument & returns Entity object if and only if the pk value exist in the table. If not exist method will return the null value, the return type is as we mentioned in 1st argument.

Ex: `Product p = em.find(Product.class, 3);`

Merge() :- It is a nonstatic method of EntityManager it will accept an Entity object which needs to be merged (save & update)

* Merge() will do update operation if & only if the primary key value of the Entity object present in the table.

* If not present the object will be saved as a new entry (or) new row.

* The return type of merge() is Generic type means whatever the type we are passing as a argument that is the return type

Ex :- product p = new product();
p.setId(5);

p.setName("pen");
p.setPrice(20);
{
Et.begin();
Em.merge(p);
Et.commit();
}

remove() :-

* It is a nonstatic method of EntityManager.
* It will accept an Entity object which needs to be deleted permanently from the database table.
* The return type of this method is void

* Since delete operation is a DML operation we have to begin & commit the transaction.

* We can't pass the id to delete directly we have to find the object 1st using find method later we can perform remove operations.

Ex :- product P = Em.find(product.class, 1);
Et.begin();
Em.remove(P);
Et.commit();

CreateQuery :-

* Non static method of Em

* It will accept a String type argument i.e. HQL

* This method is used to perform a DB operation in customized manner

* The return type of this method is Query Interface

* Query is a interface by using we can execute any custom queries it has one important methods i.e. (getResultList)

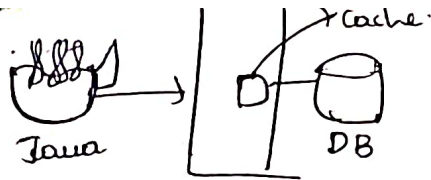
* This method is used for executing the custom queries & returns the list type reference.

~~HfBe~~

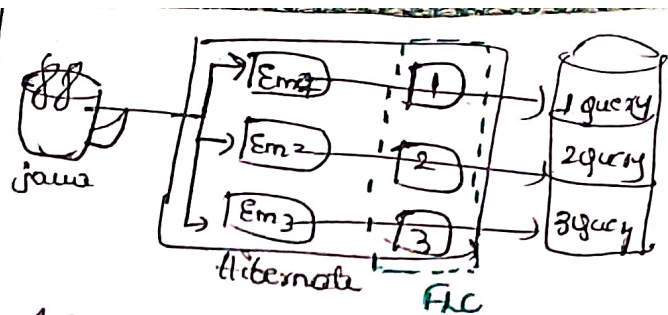
١٠

—

1



giving same id with 3 times with 1 Em → gives 1 query



1 id with 3 Em it gives 3 queries

Structure of firstlevel Cache memory (FLC)

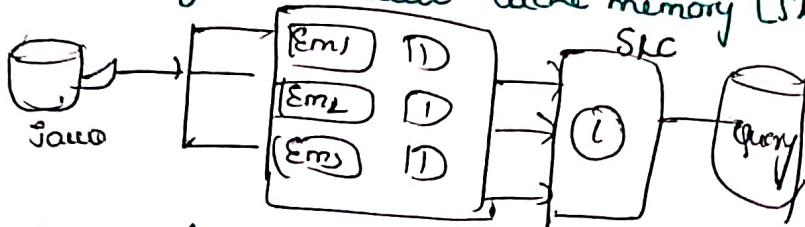
FLC (default Enable)

SLC (Explicitly Enable from repository)

- modification:-
- ① hibernate cache relocation. (5.6.15)
 - ② In Entity class we should provide @Cacheable
 - ③ GVD → Persistence.xml →
 - add persistence.xml file with 2 property.

Steps to Enable SLC

Structure of Secondlevel Cache memory (SLC)



Cache mechanism in Hibernate

- ① First level cache memory (FLC)
- ② Second level cache memory (SLC)

FLC:- It is a temporary layer of storage which is maintained & managed by ~~Every~~ Every Entity manages independently.

* default Enabled

* FLC has limited scope i.e. the job done by 1 Em can't be utilized another Entity manages.

SLC:- It is a temporary layer of storage which is common for all the Entity managers.

* Enable Explicitly.

* FLC ~~no~~ has not limited scope, the job done by 1 Em can be utilized by other Em,

* Overall performance of the application can be optimized.

Association mapping in Hibernate :-

It refers to that one object having the reference of another object is referred to as association mapping.

* Generally association mapping is categorized into four different types as below.

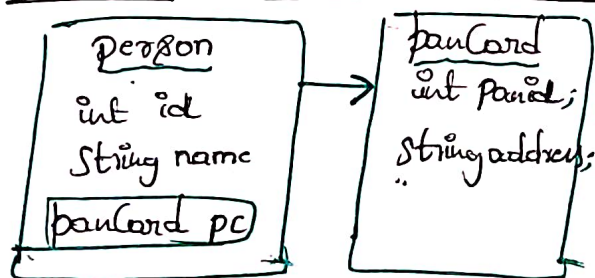
- (i) OneToOne association mapping
- (ii) OneToMany
- (iii) ManyToOne
- (iv) ManyToMany

OneToOne Association mapping :-

one object having the reference of another one object is referred to as OneToOne association mapping.

Ex:- Relationship b/w person & panCard / Aadhar / Driving licence / passport

UML Diagrammatic representation



Entity Class Design

```
public @Entity class Person {
    private int @Id @GeneratedValue (strategy=generation
                                     type=Identity) id;
    private String name;
    @OneToOne
    private PanCard pc;
}
```

ALL getter() setter()
+ no-arg constructor
+ all arg constructor

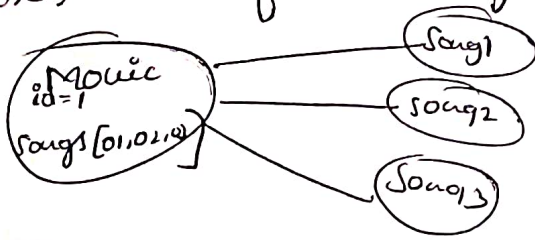
```
public @Entity class PanCard {
    private @Id @GeneratedValue (strategy=generation
                                type=Identity) int panid;
    private String address;
}
```

Note :- To achieve relationship in DB table.
To achieve relationship in java (OneToOne)

OneToMany :- one object having the reference of many other objects

refers to oneToMany relationship.

Ex:- one film & many songs



Movie	
id	

movie-songs	
mid	sid

Songs	
id	name

@Entity
class Movie {

@Id
private int id;

relationship b/w

DB-table

@OneToMany
private List<Song> songs;

↳ relationship b/w Java

@Entity

class Song {

@Id

private int sid;

private String name;

}

Note:- In one to many relationship definitely one extra table will be created by hibernate to maintain relationship b/w 2 table. and we don't have direct access to this table hibernate only handle this table.

ManyToOne :- many objects having reference of one objects.

Ex:- many states belongs to one Country etc.

@Entity
class movie {

@Id

private int id

}

@Entity

class Song {

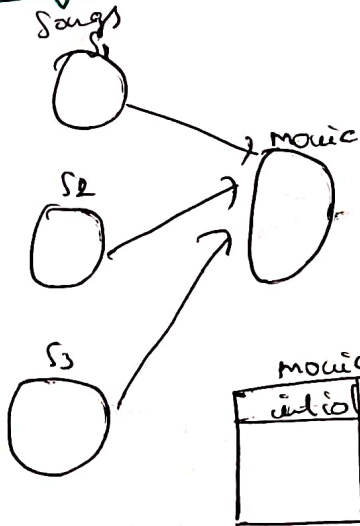
@Id

private int sid

@ManyToOne

private movie m;

Songs movie	
sid	mid

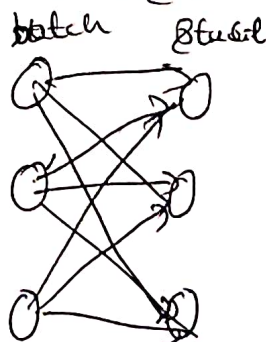
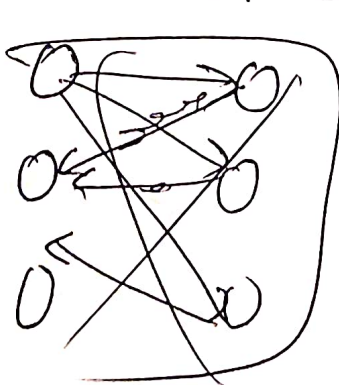


movie	
id	

Song	
id	Fk
	m

ManyToMany :- many Entity object having the reference of many others Entities references.

Ex:- batch & student, Customer & product



@Entity

class Batch.

int id;

@ManyToMany

List<Student> st

id	id

↳ with duplicate keys

class Student {

private int id

private String name;

}

Time stamp :-

Time Stamp :-

① @CreationTime Stamp :-

- * It is a field level @ belongs to Hibernate
- * This annotation will initialize the variable with the Current virtual machine date & time while persisting the object only. (It will only work for save operation)

② @updateTime Stamp

- * It is a field level @ belongs to Hibernate
- * This annotation will initialize the variable with Current Virtual machine date & time while persisting & also while merging (It will work both save & update)

Spring:-

Spring is a → open source
→ non invasive
→ light weight

→ Java based framework

used to create Enterprise applications (Business grade application)

* The founder of Spring framework Rod Johnson

* The first name of Spring framework is interface-21

* Rod Johnson has started Spring framework in 2002 & he renamed to interface-21 → Spring framework.

* It is light weight, modular, flexible to use in any applications

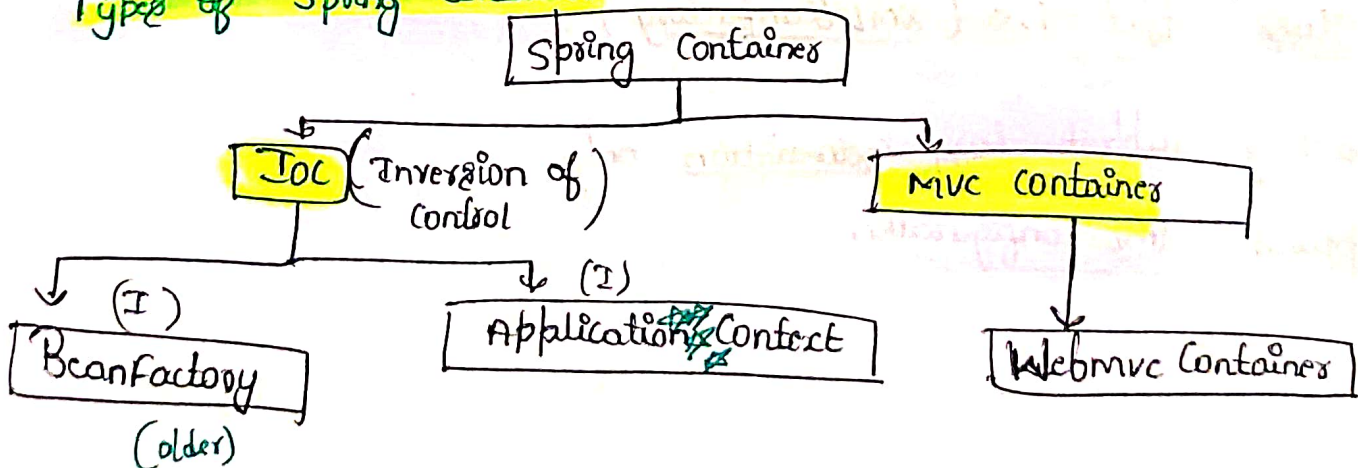
modules of framework :-

- ① Spring Core → BeanFactory
- ② Spring Context → ApplicationContext
- ③ Spring JDBC
- ④ Spring ORM
- ⑤ Spring AOP
- ⑥ Spring web, MVC

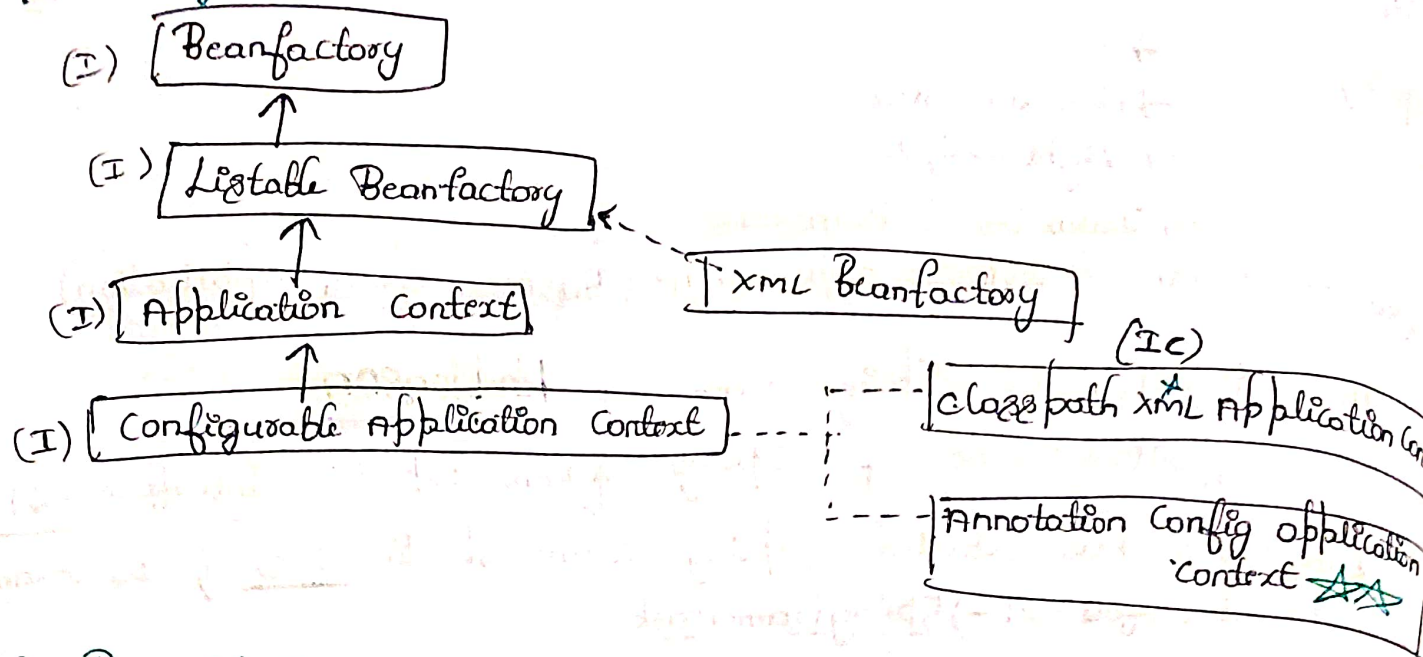
Spring Container :-

It is an engine, which will maintain and manage the entire applications and bean classes and bean objects.

Type of Spring Container.



Hierarchy of classes and interfaces in spring framework



Spring Bean class :-

A simple class for which the object will be created, initialized & maintained by the Spring Container. Such classes are called as Spring bean class & object of such classes are called as bean objects.

* Always bean object will be created by Spring Container.

@Component :- It is a class level annotation belongs to Spring framework.

* It indicates the annotated class is a Spring bean class; and objects of such classes are bean objects.

BeanFactory :- It is an interface belongs to Spring framework and it is the supermost interface of Spring framework.

* The object of bean factory can be created by an implementation class object i.e. (XML Beanfactory)

* Beanfactory support lazy instantiation only

* It supports XML Configuration

Application-Context:-

+ It is an interface belongs to Spring framework.

+ It is the subinterface of beanfactory.

+ The object of Application can be created by two ways

x1) `ApplicationContext ac = new ClasspathXmlApplicationContext();`

2) `ApplicationContext ac = new AnnotationConfigApplicationContext();`

~~Bean~~ Beanfactory

ApplicationContext

It follows lazy instantiation.

It follows Eager instantiation.

It works only with xml ^{Configurations} ~~factory~~

It can work with xml & annotated class

It is Core container (by ^{supremacy} ~~interface~~)

It is called as JEE _{2.1 version} container

Inversion of Control (IoC)

The process of transferring the object creation control from developers side to Spring container side.

Dependency Injection:-

The process of injecting the object address into the reference variable. (default value is there but object address should not null)

Types of Dependency Injection:-

1. Setter injection.
2. Constructor injection.
3. Field injection.

Autowired :- The process of achieving the dependency injection by Spring container at the runtime.

- Types :-
1. Autowiring by fields.
 2. Autowiring by Setter.
 3. Autowiring by Constructor.

```

class Product {
    @Autowired
    Laptop l;

    Product() {
        s.o.pln("Hi");
    }
}

```

product.java

```

class Laptop {
    Laptop() {
        s.pln("Bye");
    }
}

```

Laptop.java

```

class Test {
    psvm() {
        AnnotationContext ac = new AnnotationConfigApplicationContext("config.class")
        Developer d = ac.getBean(Class.Product) → address of product
        s.o.pln(d.l); → address of Laptop class
    }
}

```

```

@ComponentScan("package name")
class Config {
    }
}

```

Autowiring through fields:-

* It is the process of achieving dependency injection directly through the fields ^(DI) called as autowiring through field (by using autowired).

Autowiring through setter:-

* It is the process of achieving DI directly through setter.

Autowired through constructor

* It is the process of achieving the DI through parameter constructor.

- Note:-
1. for Constructor Injection parameterized Constructor is mandatory.
 2. In real time projects in most cases the preferred way of dependency Injection is Constructor Injection only.

@Autowired:-

- * It is a field level, method level, constructor level annotation belongs to Spring framework, which is used to achieve autowiring.
- * If we use this annotation over a field, field injection will happen, Setter $\rightarrow$ Setter Injection, constructor (parameterized) $\rightarrow$ constructor Injection.

@ComponentScan:-

- * It is class level annotation, it is used to scan for the component ^{classes} which is mentioned in the package.

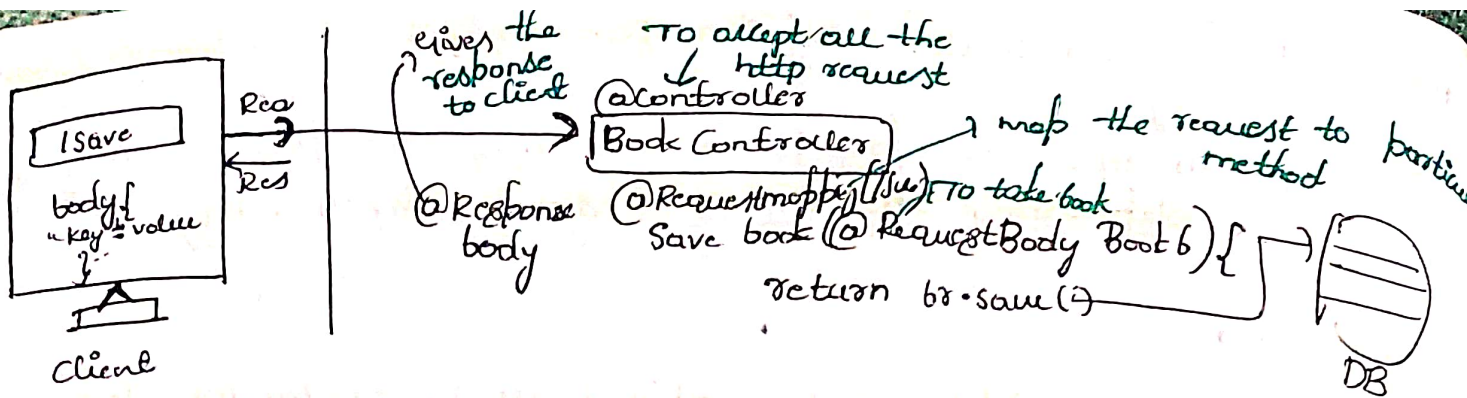
Ex:- `@ComponentScan("packagename")`

@GetBean():- It is a nonstatic method of Spring Application Context Interface.

- * It is used to get the bean address (object) objects.
- * This method will accept 1 argument, i.e. the type of the bean which we want.

Ex:- `Developer d = ac.getBean(Developer.class)`

- * The return type of `getBean()` ~~is~~ is generic type means whatever the type of argument that is the return type.



@ResponseBody :- It is class/method level annotation

@RestController :- @Controller + @ResponseBody
Class level annotation, it is the combination of

@RequestMapping :- It is the generic method type (drawbacks)

It is generic mapping annotation
It will not check the method types
So use the → GetMapping ("/") for GET
→ PostMapping ("/") for Save (POST)
→ DeleteMapping ("/") for DELETE

POST → Save
PUT → Complete update
DELETE → delete
PATCH → partial update
GET → fetch

RequestMapping is a generic mapping annotation, it will not check for the method type use

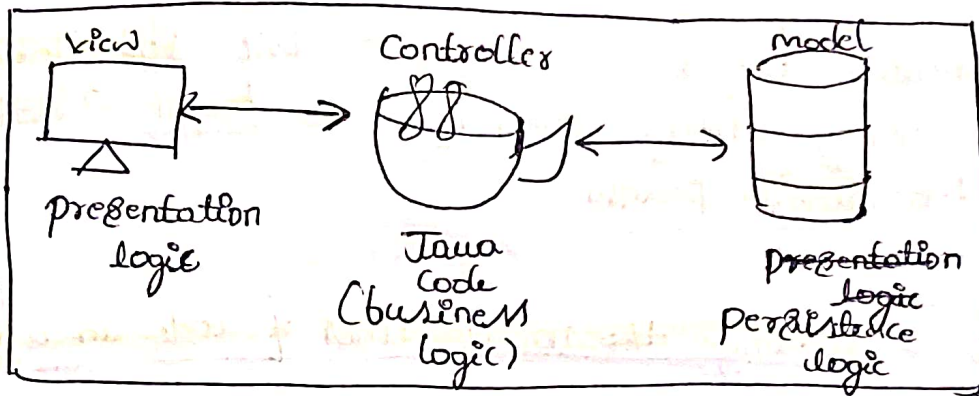
- * GetMapping ("/")
- * PostMapping ("/")
- * DeleteMapping ("/")

Spring MVC :-

MVC stands for model view Controller

- * It is a special module to create web applications.
- * MVC application is integrated with presentation logic, persistence logic, business logic.
- * Model refers to data, view refers to (presentation logic), Controller refers to Java code

Diagrammatic representation



- * We created MVC application using Spring MVC module but while developing the application we faced lots of technical problems some are as below,

Limitation / drawbacks of Spring MVC

- * Spring MVC needs boiler plate configuration which is ^{mandatory} should be done by developer only. (e.g. `mysql-service.xml`)
 - * In Spring MVC we have to maintain server explicitly which may require extra attention of developer to maintain the server.
 - * It ~~serves~~ serves only web application as a client but it can't serve mobile client such as android / ios / windows client & many more drawbacks are there so...
- To overcome this we go for Springboot

Springboot :-

- It is a Java based framework to create Enterprise applications.
- * It is an extension of Spring framework where all the configuration of Spring framework will be automatically done in Springboot.
- * Springboot web applications internally maintain the server so developers no need to maintain the server.
- * It works on a principle called Convention Over Configuration
↳ without any configuration just write code.
- * To create a Springboot project we have to use SpringInitializer web application. SpringInitializer simplifies the project creation & adding the dependencies process.

* Springboot is aimed to minimize the no. of lines of code in an app.

* Springboot is widely used to create REST APIs.

REST API :-

* REST stands for Representation States transfer.

* REST API is a one using which we can transfer the state of an object in a representable format b/w two software systems such APIs are called as REST API's.
Ex: object save → DB
 client

^{1a} * Springboot object representation format used is JSON.

* JSON stands for JavaScript object notation. In JSON we can represent an object by $\{ \text{"key"} : \text{value}; \}$ and states of an object as key value pairs.

Ex: $\{ \text{"name"} : \text{"Rishma"}, \}$
 $\text{"Age"} : 22$
 $\}$

Note:- always key should be "" strings in json. (Enclosed in "")

Important annotations of Spring Boot

① @SpringBootApplication

- It is the most important annotation of Spring Boot application.
- It is class level annotation belongs to Spring Boot.
- It indicates the annotated class is the starting point of Spring Boot applications.

→ It is a combination of $\left\{ \begin{array}{l} @Component, @EnableAutoConfiguration \\ @SpringBootApplication \end{array} \right\}$

→ also In Spring Boot ApplicationContext is used as Spring Container.

② @Controller:- It is a class level annotation belongs to Spring Framework.

- * It indicates that the annotated class is a Controller class which is responsible to accept all the http request (client request).

③ @RequestMapping:-

- * It is method level and class level annotation belongs to Spring Framework.
- * It is used to map the specific http request to specific method.

Syntax:- @RequestMapping (" / endpoint ")

④ @RequestParam:-

- * It is a field level annotation belongs to Spring Framework.
- * It is used to map the request parameters to method parameters.

ex:- localhost:8080/findByid?id=10 + url

map → public String (@RequestParam int id)

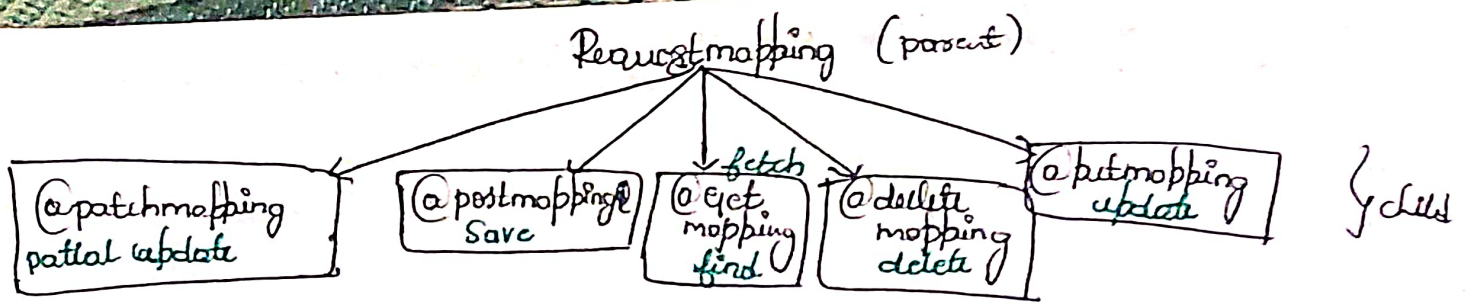
Note:- always @request parameter is same the method parameters.

⑤ @RequestBody:-

- * It maps the request body to method parameters.

{ "name": "Rishu",
"age": 18
}

 → public Person (@RequestBody Person p)



Note:- 1. Request mapping is a generic mapping annotation which doesn't check for method types.

2. we have specific mapping annotation to check for method types as above mentioned

⑥ @PathVariable:- It is used to map the ^{from} data in path variable to method parameter.
 Example: `@GetMapping("/find/{id}")` → localhost:8080/find/40
`public String getBook(@PathVariable int id)`
 → It is used to map the data to the method parameters from the url path.

JPA Repository:-

It is an interface belongs to Springboot framework which has the methods to perform database operation (CRUD)

important methods of Jpa repository:-

- 1) Save ()
- 2) deleteById ()
- 3) findAll ()
- 4) ~~deleteBy~~ findById ()

Save():- It is a non static method of Jpa repository it is used to save the Entity object.

* this method will do save operation, if we are passing an Entity object without id.

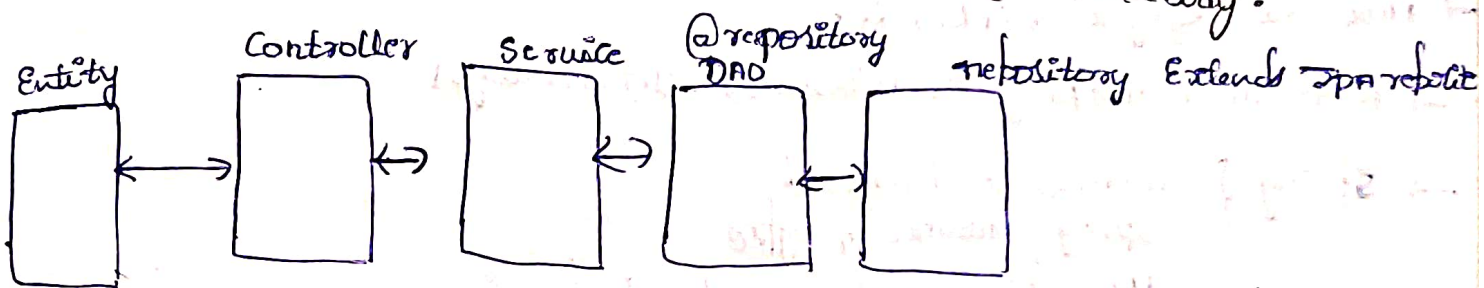
* this method will do update operation, if we are passing an Entity object with id.

② findAll():- It is the nonstatic method of JPA Repository.
It is used to select all the objects from the db table.

③ findById():- It is the nonstatic method of JPA repository.
It is used to fetch the object based on id from the db.

④ deleteById():- It is a nonstatic method of JPA repository.
* It is used to delete the Entity object based on id from the db.

⑦ RestController:- It is the class level annotation belongs to Spring framework.
* It is the combination of @Controller and @ResponseBody.



@Response Entity → all the methods types becoz we should maintain the industry standard

```
return responseEntity.status(200).body(

{ status code  
msg  
body  
} )


```

response structure

Questions :-

- JDBC steps
- JDBC drawbacks
- Servlet, lifecycle
- Frameworks, types, pros & cons
- ORM, specification of ORM, @Entity, @Id, Rules to create Entity class
- Hibernate { advantages, structure of hibernate, JPA }
- HQL { steps { emf, emf, et } methods
- Hibernate lifecycle, { FLC, SLK }
- Association mapping in Hibernate, disadvantages
- Spring { modules of framework.
Spring container & types
Hierarchy & class & interface of Spring.
- Difference b/w Application context bean factory
- Spring bean class { @Component }, @ComponentScan, getBean
- IOC, Dependency Injection types (autowired),
- Spring MVC, disadvantages
- Springboot { Rest APIs, aim
Convention over Configuration.
- Annotation :-
 - @SpringBootApplication → @Configuration, @SpringFactories, @EnableAutoConfiguration
 - @Controller, @RestController → @RestController
 - @RequestMapping → generic
 - @RequestParam
 - @RequestBody
 - @PathVariable
 - RequestBody